

```
> ./how-much-work-is-it --profile=rva23
```

Supporting a New CPU's ISA Extensions in the Linux Kernel

— how much work is it, really?

RVA23 in Linux: runtime detection & compile-time specialization



Guodong Xu

Qualcomm · Linux kernel contributor
RISC-V International Advocate

> whoami



Guodong Xu

Qualcomm Linux Kernel Team

- 12 yrs at Linaro — building the Arm ecosystem
- RVA23 enablement in mainline Linux
- RISC-V International Advocate

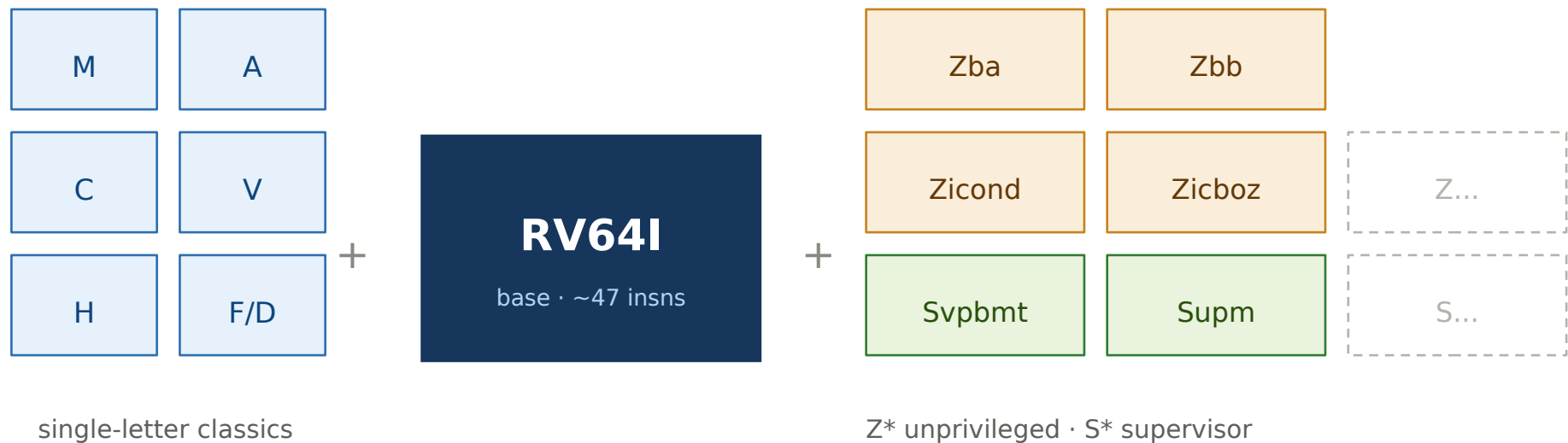
I'm a Linux kernel contributor.

docularxu.github.io



> risc-v 101 — a modular, open ISA

One small base + everything else is an extension.

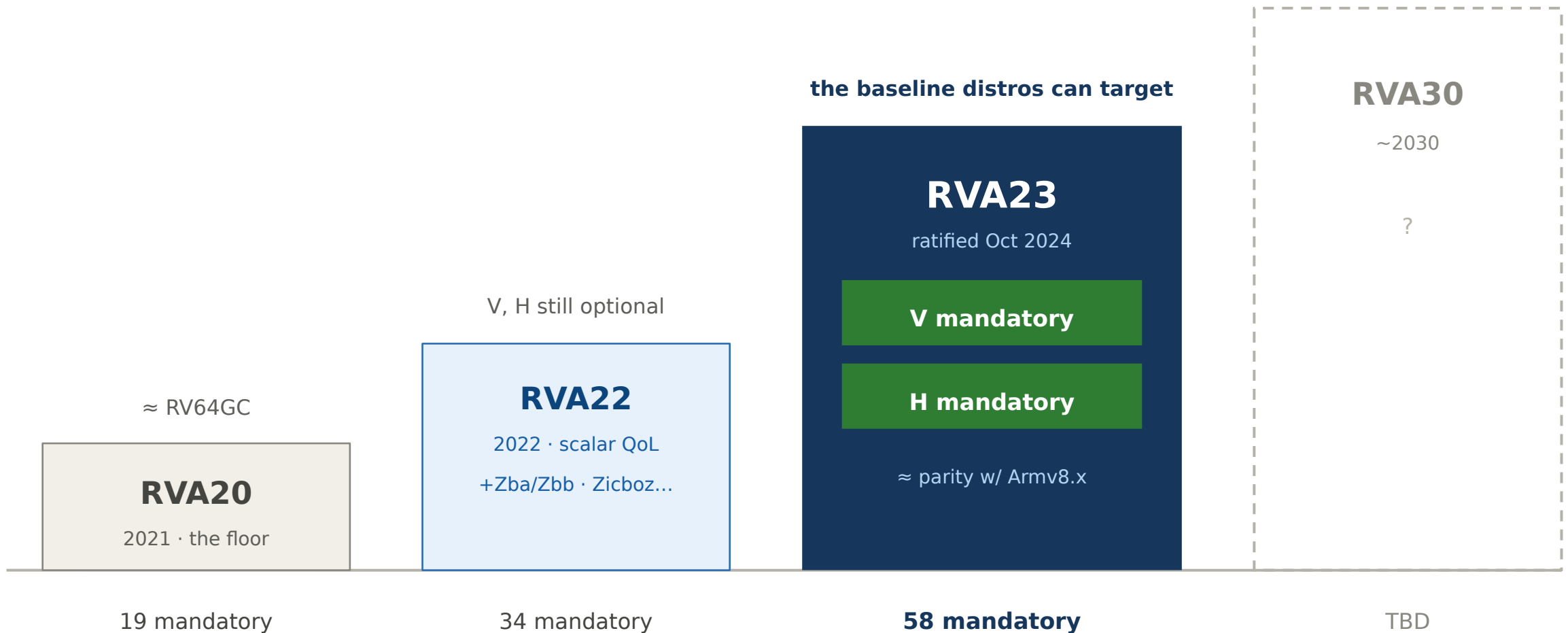


- open standard — no license fee, anyone can implement
- an extension becomes standard through ratification

x86 / Arm: monolithic ISA + version numbers. RISC-V: Lego.

> profiles — from wild west to baselines

Freedom to compose → fragmentation. Profiles are the fix.



> diff rva22 rva23 — what the additions add up to

+ vector: V (RVV 1.0) · Zvbb · Zvfhmin · Zvkt

SIMD everywhere, at last

new

+ hypervisor (S64): Sha = H + Sh* · Sstateen

KVM on every conformant chip

+ security: Supm · (Zvkt timing)

HWASan-class tooling ≈ Arm TBI/MTE

+ scalar: Zicond · Zcb · Zfa · Zawrs · Zimop/Zcmop

tighter codegen, fewer branches

RVA22 base — 34 inherited

RV64GC · Zba/Zbb/Zbs · Zicbo* · Zic64b · Zkt ...

floor

What RVA23 is:

the profile where RISC-V
becomes application-class

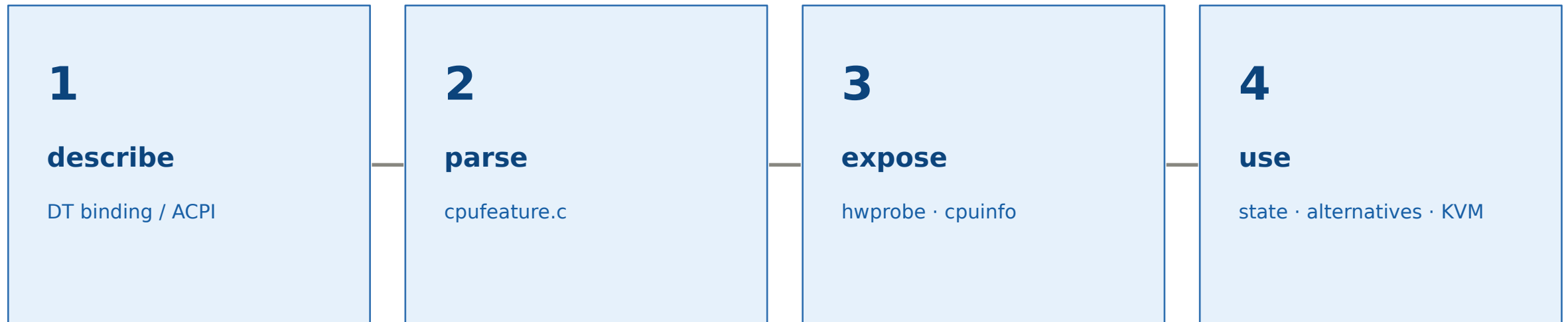
- SIMD you can rely on
- VMs you can rely on
- sanitizers you can rely on

≈ Armv8.x baseline parity

phones · servers · distros

+24 newly mandatory → 58 total. CPU-ISA only; AIA / IOMMU live in platform specs.

> so, how much work is one extension?



> the one question that sets the cost

Does it add state the OS must save/restore?

no — stateless

insn-only

21

B · Zicond · Svinval
new insns, old regs

property

20

Zic64b · Sv39 · Zicc*
a guarantee, not a feature

yes — stateful

fp

4

F · D · Zfa
mstatus.FS

vector

4

V · Zvbb
mstatus.VS

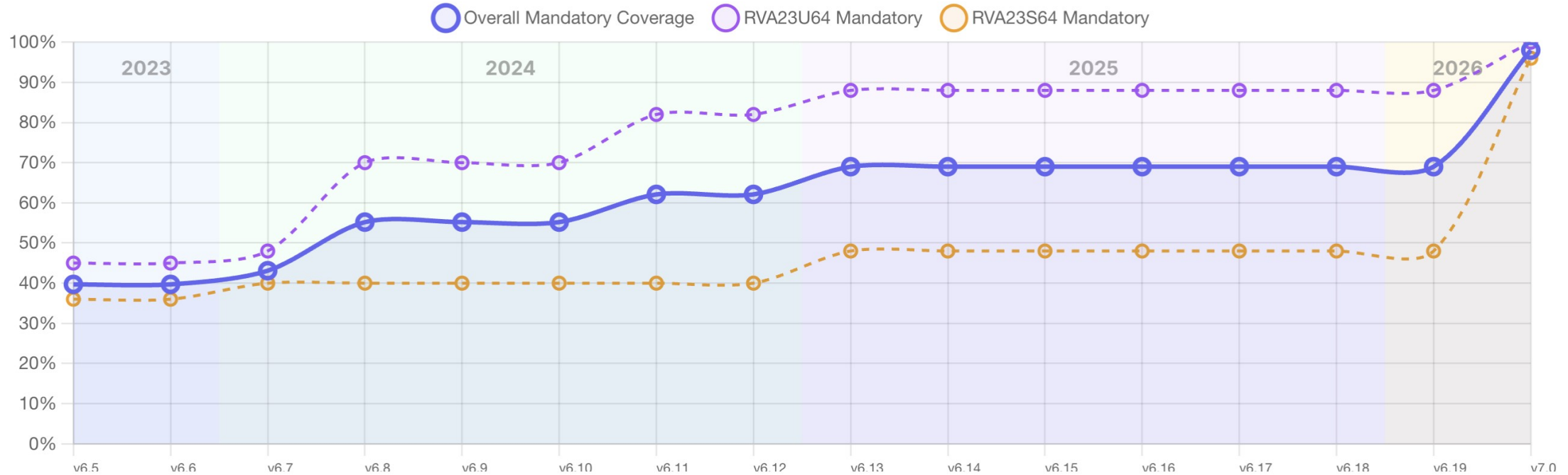
csr-only

9

H · Supm · Sstc
Smstateen-gated

**1/3 of RVA23 is property bits: zero kernel logic —
but invisible to everyone unless the kernel reports them.**

> reality check — my first RVA23 chip upstream (SpacemiT K3)



~68%

stuck ~1 year (v6.13 → v6.19)



100%

v7.0-rc1 · Feb 2026

"Is the description incomplete,
or is the CPU not compliant?"

— Heinrich Schuchardt (Canonical)

submit, don't strip · +18 → extensions.yaml · deepest nail: Supm

> kernel: done. distros: ... ?

wait-and-see

Debian

RV64GC · Forky '27: RVA20 at most

Fedora

RV64GC · SIG paused RVA23

Red Hat

dev preview · no commitment

SUSE

no RISC-V SKU at all

the only two on RVA23

Ubuntu 26.04 LTS ✓

hard cutover · first LTS to mandate RVA23

openEuler 24.03 SP3 ✓

dual-track RVA20+RVA23 · backport-heavy

+ openRuyi rolling testbed · ~3,000 pkgs on -march=rva23u64

but x86-64 already solved this — glibc-hwcaps

SLES 16: 30+ libs auto-dispatched above the baseline (openssl, libgcrypt, ...)

baseline stays put → optimized libs load at runtime

RISC-V's missing piece: capability detection.

58 extensions don't fit a HWCAP bitmask → hwprobe

> two roads — both needed, no merge

kernel knows RVA23

```
graph TD; A[kernel knows RVA23] --> B[detect at runtime]; A --> C[compile as RVA23];
```

detect at runtime

one kernel, every CPU

→ /proc/cpuinfo · hwprobe

for: distro kernels

portability > specialization

compile as RVA23

one CPU class, faster kernel

→ -march=..._zba_zbb CFLAGS

for: purpose-built targets

performance > portability

> the runtime road — my hwprobe series (v5, in review)

RVA23U64 conformance: one call, not 58 probes.

```
$ cat /proc/cpuinfo
...
isa bases : rv64ima rva23u64
...

hwprobe → BASE_BEHAVIOR_IMA | RVA23U64

K3 silicon + QEMU rva23s64 · v7.2-rc1
```

17 patches · 5 authors

Guodong Xu	10
Andrew Jones	4
Charlie Jenkins	1
Jesse Taube	1
Qingwei Hu	1



Aurélien Jarno (Debian riscv64): a good fit for RVA23-optimized libs via glibc-hwcaps

> the compile-time road — build the kernel as RVA23

Charlie Jenkins — RVA23 series (RFC)

```
CONFIG_RISCV_ISA_RVA23=y  
CFLAGS += -march=..._zba_zbb_zicnd...  
  
# gcc now emits these in plain C —  
# no asm, no alternatives
```

LKML Dec 2025 · Plumbers 2025

Paul Walmsley — the review gate

"...these just enable building
an RVA23-specific kernel."

IS THIS A WIN?
show hardware numbers

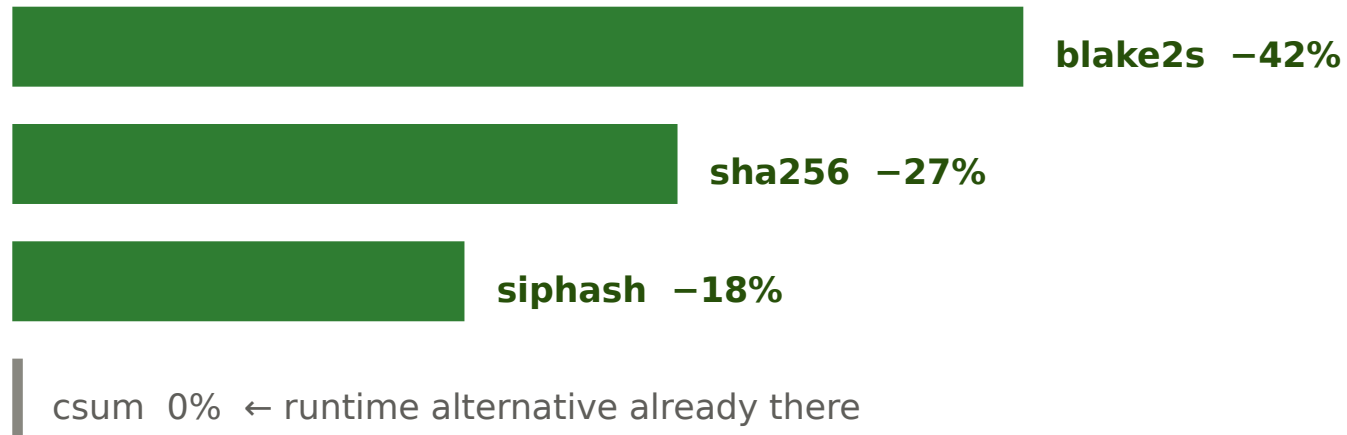
Answering that needs real RVA23 silicon.

2024: Spike/QEMU only → Apr 2026: K3 ships → May 2026: measured

> measured on RVA23 silicon (SpacemiT K3 · X100)

A = RVA23 build (Zba/Zbb in CFLAGS) vs B = same .config, without

per-primitive speedup



whole kernel

-2.0%

static instructions, whole vmlinux

+68,054

Zba/Zbb insns the compiler emitted

csum's 0% is the point: where runtime alternatives exist, both roads meet.

the win lives in plain C code no one hand-optimized

cross-arch reproduced on K1 X60 (non-RVA23): dd -0.44% insns · csum 0% — Paul's matrix 3/3 · full data: backup

> what's next

Server Platform v1.0

ratified May 2026

AI matrix extensions

IME

AME

VME

Tested-by: · Reviewed-by: — they count.

Thank you.



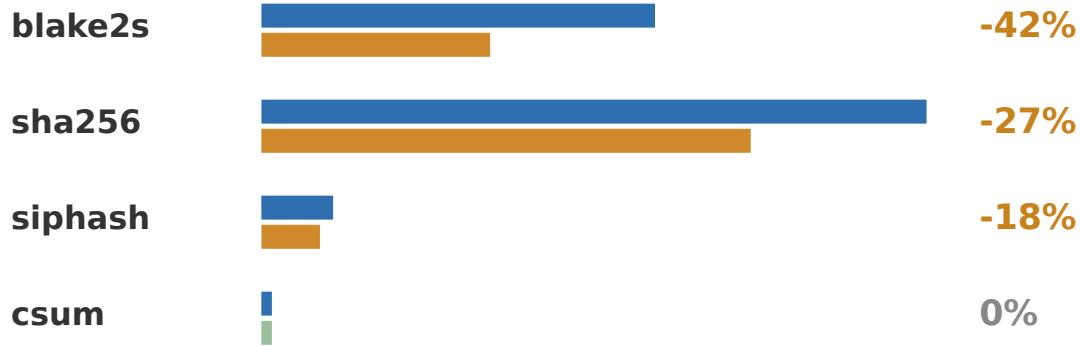
THE NUMBERS (backup)

Compile-time Zba/Zbb on K3 X100

A = RVA23-compiled

Zba/Zbb in CFLAGS → compiler emits them in plain kernel C

FASTER · ns to hash 4 KiB · shorter = faster



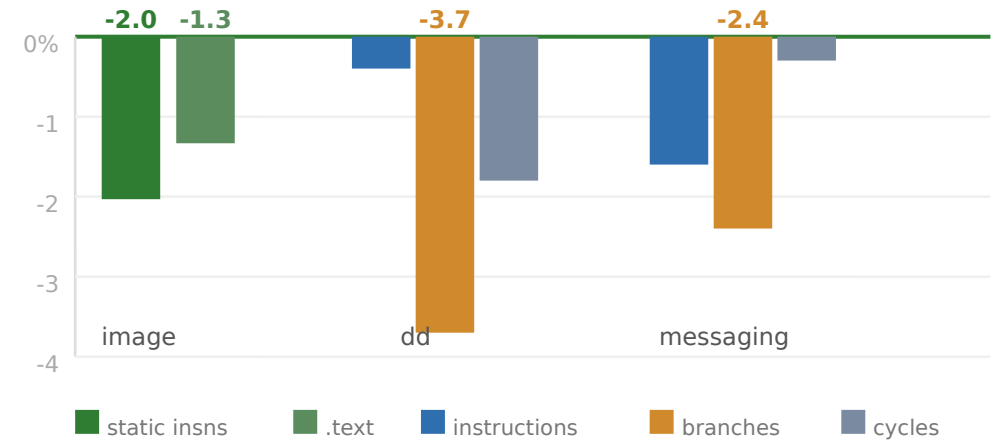
csum already has a runtime Zbb path → 0%. Wins live in pure-C primitives.

✓ cross-uarch: K1 X60 reproduces dd -0.44% / -3.11%

B = vanilla baseline (control)

same .config — only difference: Zba/Zbb NOT in CFLAGS

SMALLER · % reduction (A vs B) · down = less



WHY IT SHRINKS · static codegen, whole vmlinux (A vs B)

+68,054

compiler Zba/Zbb ops in kernel C

1 Zbb op

1 Zbb op = 3 base ops

~ 2.3 base ops → fewer, denser

blake2s_compress_generic (insns)

B 2018

A 1368

-32% (matches the -42% dynamic)

same math on X60: dd -0.44% · siphash -37.8% (narrower pipe → bigger win)